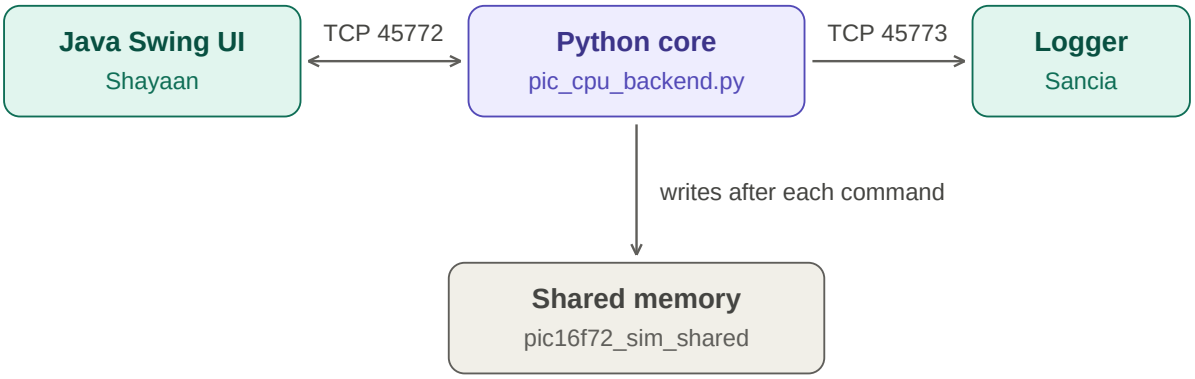
PIC16F72 Simulator: IPC Architecture

Java Swing UI, Python core and logger. Team: Mohammed Ayman, Mohammed Shayaan, Prakrithi, Sancia.

1. Overview

The simulator is three cooperating programs. The Java UI sends commands to the Python core over a local TCP socket. The core sends events to a logging process over a second socket, and publishes the CPU state in shared memory.

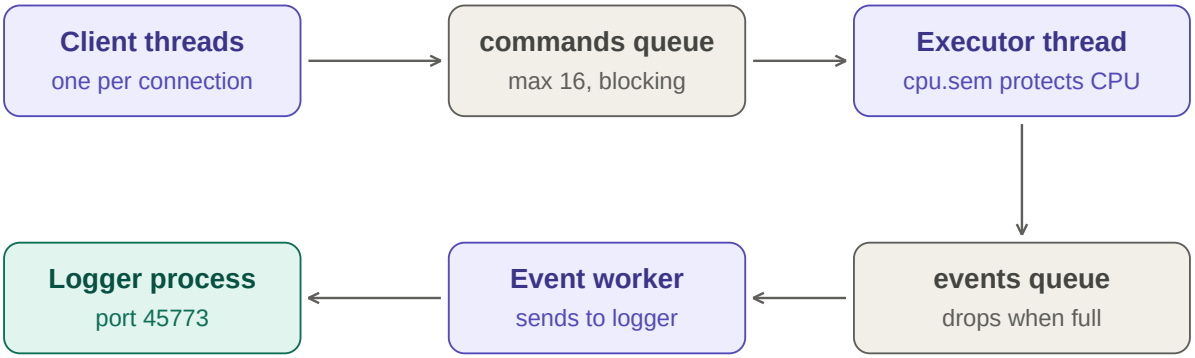


2. IPC channels

Channel	Mechanism	Name or port	Direction	Used for
UI and core	TCP socket, localhost	127.0.0.1:45772	Both ways	Commands LOAD, STEP, RESET, QUIT. Each reply ends with a line ---END---.
Core to logger	TCP socket, localhost	127.0.0.1:45773	Core to logger	One short connection per event message (for example EXECUTED, HALTED, RESET).
CPU state	Shared memory	pic16f72_sim_shared, 84 bytes	Core writes	W, PC, Z flag, C flag, halted and memory 16 to 31. Only the executor thread writes.

3. Inside the Python core

A command passes through two queues. Client threads put commands on the commands queue, and a single executor thread runs them one at a time. The executor puts events on the events queue, and the event worker forwards them to the logger.



4. Synchronization

Part	Role	How it is synchronized
Client threads	One per UI connection. Read command lines and queue them.	commands queue (max 16). put() waits when the queue is full.
Executor thread	Runs each command on the CPU and sends the reply.	cpu.sem, a Semaphore(1) used as a lock, protects the CPU state during step, load, reset and publish.
Event worker thread	Prints events and sends them to the logger.	events queue (max 16). A full queue drops the event, so logging never blocks the CPU.
Stop event	Shutdown flag set by QUIT and at exit.	All loops check it. Queue reads time out every 0.2 s.

Known limitation: the shared memory write has no cross-process lock, so a reader in another process could briefly see a half-updated state.

Source: pic_cpu_backend.py. The Java UI side is described from the protocol the Python core expects.